

Phase 3 - Shared Redis Cache

Previous vs new implementation comparison | Repository: Office50505/Edu-nex_Final | Backend: edunex-b | 19 September 2026

PHASE 3 CODE STATUS	SHARED CACHE	TESTS (FOCUSED)	READY TO DEPLOY
PASS	REDIS ONLY	7 / 7	YES

What changed

Area	Previous implementation	New implementation
Production fallback	Redis connection, read, or write failures silently used nested process-local Map caches.	When REDIS_URL is configured, reads become misses and writes/invalidation become safe no-ops. No local replacement is created.
Backend status	Reported redis whenever REDIS_URL existed, even before a successful connection.	Reports redis-connected only when the single client is ready; otherwise redis-disconnected. Production without a URL reports disabled/no-shared-cache.
Connection policy	One client, but default reconnect behavior and an isOpen check could queue work while reconnecting.	One client per process, 2-second connect timeout, bounded exponential reconnect (3 retries, max 2 seconds), handled error/ready events, and immediate miss while reconnecting.
Logging	Logged raw error messages, which could theoretically contain connection details.	Logs a rate-limited operation label and allowlisted error code only. URLs, passwords, and raw error messages are never emitted.
Development	Memory fallback was available whenever Redis was unavailable.	Memory cache remains available only when REDIS_URL is absent and NODE_ENV is not production.

Shared cache coverage and TTLs

Namespace	Data	TTL	Invalidation
public-read	Categories; anonymous recommendations; published course list	60 seconds	Category create clears public-read. Course create/update/delete clears public-read and checkout-summary.
checkout-summary	Featured course; course:v3:<id> summaries	600 seconds	Course create/update/delete scans and deletes the shared namespace in Redis.

Key namespace: Existing CACHE_PREFIX support and the default edunex application prefix are preserved, so no Redis flush is required. Keys retain the format <prefix>:<namespace>:<key>.

Intentionally local: Admin IP-location cache, Bunny pull-zone discovery, 10-second admin diagnostics, and provider access-token caches were not moved. Rate limiting was not changed or enabled.

Failure behavior

Event	Application behavior
Redis GET/connect failure	Return a cache miss, query MongoDB/source data, and continue the response where the authoritative source is available.

Event	Application behavior
Redis SET failure	Log safely once per outage period and continue. The fetched result is not stored in process memory.
Redis delete/invalidation failure	Log safely and continue the write request. No local-only invalidation is presented as shared invalidation.
Redis reconnecting	Do not queue cache commands behind a non-ready client. Treat the request as a miss until the client is ready again.

Files changed

File	Change
edunex-b/services/cacheService.js	Redis-only configured mode, bounded connection behavior, safe logging, real readiness status, and development-only memory mode.
edunex-b/tests/shared-cache.test.cjs	New focused tests for shared reads/writes, TTLs, source fallback, invalidation, status, secret protection, and environment policy.
edunex-b/package.json	Added the test:cache command. Redis dependency remains redis 6.0.0 and is present in package.json and package-lock.json.

Verification

Check	Result	Notes
Syntax and diff validation	PASS	node --check for service and tests; git diff --check clean.
Focused cache suite	7 passed / 0 failed	npm run test:cache
Relevant cache/status/course suite	14 passed / 0 failed	Shared cache, admin course list, and system health tests.
Complete backend suite	176 passed / 1 failed	Only failure: local node_modules lacks already-declared @aws-sdk/client-s3 for the unrelated thumbnail-CDN test. No thumbnail/S3 code was changed.

Acceptance result

Shared public cache	PASS
Shared checkout cache	PASS
Redis invalidation	PASS
Production local fallback removed	PASS
Redis failure -> cache miss	PASS
MongoDB/source fallback behavior	PASS
Development memory fallback	PASS
Secrets protected	PASS

Risks and deployment note

Live ElastiCache was not contacted from this workspace; rediss:// handling was verified with a mocked TLS URL. Redis outages intentionally increase MongoDB/source load because no local fallback is created. Invalidation uses incremental SCAN batches, not a key flush. After deployment, install locked dependencies and confirm redis-connected after the first cache operation. No deployment or changes to AWS, PM2, frontend, rate limiting, billing, jobs, S3, or MongoDB were made.